Índice

1. Introducción....1
2. Funciones predefinidas del lenguaje....2
Diferencia entre encodeURI y encodeURIcomponent :....3
3. Funciones del usuario....3
3.1 Definición de funciones....4
3.2 Invocación de funciones....5
3.3 Consideraciones sobre funciones....6
4. Arrays....10
4.1 Declaración de Arrays....10
4.2 Inicializacion de Arrays....11
4.3 Arrays y bucles....11
4.4 Propiedades de los Arrays....11
4.5 Métodos de los arrays....13
4.6 Arrays multidimensionales....16
5. Objetos definidos por el usuario....17
5.1 Objetos Simples....18
5.2 Objetos Literales....18
5.3 Funciones como constructores....18
5.4 Agregando Métodos....19
5.5 Herencia....20
5.6. Objetos json y arrays....23
5.7 Ejemplos....25
6. Clases en javascript....30

1. Introducción.

En cualquier lenguaje de programación se suelen repetir partes de código que realizan una serie de tareas, por ejemplo, comprobar el tipo de datos que ingresa un usuario por pantalla, controlando si los datos son númericos o cadenas de texto. Para realizar estos controles debemos definir una serie de instrucciones que pueden ocupar bastantes líneas de código. Cuando un bloque de instrucciones se repite una y otra vez en puntos distintos de un programa, el código fuente de la aplicación se complica por :

- El número de líneas de código aumenta ya que muchas de éstas estarán repetidas.
- El mantenimiento se dificulta, puesto que si deseamos modificar una instrucción de las que se repiten debemos hacerlo en todos los puntos de la aplicación donde se encuentre ese bloque de

instrucciones.

Javascript, al igual que en todos los lenguajes de programación, permite agrupar ese bloque de código que se repite e invocarlo cuando lo necesitemos. Esta serie de instrucciones englobadas dentro de un mismo proceso se denominan **funciones**. En esta unidad veremos las principales funciones predefinidas por javascript, además de como crear y utilizar nuestras propias funciones.
En esta unidad también veremos los arrays como un elemento en la gestión eficaz de datos.

En la última parte de esta unidad veremos como crear objetos definidos por nosotros mismos, con sus métodos y propiedades.

2. Funciones predefinidas del lenguaje.

Las funciones predefinidas de Javascript están integradas en el lenguaje. Podemos utilizarlas sin tener que declararlas previamente. Solamente tenemos que conocer el nombre de la función y el resultado final que obtendremos al utilizarla. La mayoría de las veces las funciones deben de acceder a valores de variables para trabajar sobre ellas. Estas variables reciben el nombre de argumentos o parámetros de la función.

escape () encodeURI()	Se utiliza para codificar URI. La función escape() está obsoleta y es mejor utilizar : encodeURI() y encodeURIComponent() http://www.w3schools.com/jsref/jsref_encodeuri.asp
eval()	Convierte una cadena que pasamos como argumento en código JavaScript ejecutable.
isFinite ()	Verifica si el número que pasamos como argumento es o no un número finito.
IsNaN ()	Comprueba si el valor que pasamos como argumento es un de tipo numérico
Number ()	Convierte el objeto pasado como argumento en un número que represente el valor de dicho objeto.
String()	Convierte el objeto pasado como argumento en una cadena que represente el valor de dicho objeto.
parselnt()	Convierte la cadena que pasamos como argumento en un valor numérico de tipo entero.
parseFloat()	Convierte la cadena que pasamos como argumento en un valor numérico de tipo flotante.

Diferencia entre encodeURI y encodeURIcomponent :

Usar encodeURI() cuando deseemos trabajar una con URL. Si hacemos :

```
encodeURI("http://www.google.com/a file with spaces.html")
```

Obtenemos :

```
http://www.google.com/a%20file%20with%20spaces.html
```

Si llamamos a encodeURIComponent destrozaremos la URL y tendremos :

```
http%3A%2F%2Fwww.google.com%2Fa%20file%20with%20spaces.html
```

Usar encodeURIComponent cuando deseemos codificar una URL como parámetro :

```
param1 = encodeURIComponent("http://xyz.com/?a=12&b=55")
```

Ahora podemos crear la URL que necesitemos :

```
url = "http://domain.com/?param1=" + param1 + "&param2=99";
```

Obtendremos la URL completa :

```
http://www.domain.com/?param1=http%3A%2F%2Fxyz.com%2F%Ffa%3D12%26b
%3D55&param2=99
```

Puedes probar el siguiente código :

```
var cadena = prompt("Teclea una cadena");
alert("Cadena codificada" + encodeURI( cadena ) ) ;
```

Ejemplo eval()

```
var operacion = prompt("Teclea una operación numérica como 2 + 2, etc ");
alert("El resultado es : " + eval( operacion ) );
```

3. Funciones del usuario

Aparte de las funciones que integra el lenguaje Javascript, podemos crear nuestras propias

funciones para llevar a cabo las tareas que necesitemos. Se puede ver una función como un conjunto de instrucciones agrupadas bajo un nombre.

3.1 Definición de funciones.

```
function nombre( [parametros] )
{
        grupo_de_instrucciones;
        [ return valor; ]
}
```

Una función se define con la palabra clave function seguido del nombre de la función y los paréntesis. Puede tener o no parámetros. Luego, entre llaves, un conjunto de instrucciones y opcionalmente la palabra clave **return** y el valor de retorno.

El nombre de la función debe cumplir una serie de reglas como que no puede empezar por número, no debe ser una palabra reservada del lenguaje o que no debe ser un nombre de función repetido.

Argumento : Ya hemos utilizado funciones con o sin argumento. Por ejemplo, cuando llamo a la función – método - alert("hola"), se le pasa un parámetro – en este caso una cadena de texto – que es lo que la función alert va a mostrar. La función – método - random(), no recibe ningún parámetro. Actúa sin necesidad de que le pasemos ningún valor.

return : Hay veces que la función hace algo, como un alert, y no necesita devolver ningún valor. Muchas veces una función devolverá un valor para que la instrucción que llamó a la función pueda trabajar con ese valor. Esto se realiza con la palabra clave **return** seguida del valor a devolver.

Ejemplos :
Función que no recibe ningún parámetro y no devuelve nada.

```
function mensaje() {
        alert("Bienvenido a mi página web \n Espero que disfrute mucho ");
}
```

Función que recibe un parámetro y no devuelve nada.

```
function mensaje( valor ) {
        alert("Bienvenido a mi página web \n " + valor ) ;
}
```

Función que recibe un parámetro y devuelve un valor.

function dameIva(cantidad) {
// Devuelve la cantidad pasada con el IVA
return cantidad * 1.21 ;
}

3.2 Invocación de funciones

Una vez que la función está definida, necesitaremos llamarla para que el navegador ejecute el grupo de instrucciones que que realiza. Para ello llamamos a la función por su nombre seguido de paréntesis y los parámetros (si los hay) en el mismo orden en el que fueron definidos.

. Invocando a una función desde Javascript.

```
<html>
<head>
<title>Invocar función desde JavaScript</title>
<script type="text/javascript">
function mi función() {
//instrucciones
}
</script>
</head>
<body>
<script type="text/javascript">
mi_funcion();
</script>
</body>
</html>
```


```
<html>
<head>
<title>Invocar función desde JavaScript</title>
<script type="text/javascript">
function mi función() {
//instrucciones
}
</script>
</head>
<body>
<script type="text/javascript">
       mi_funcion();
</script>
</body>
</html>
```


. Invocando una función desde HTML

```
<html>
<head>
```

```
<title>Invocar función desde JavaScript</title>
<script type="text/javascript">
function mi_funcion(){
//instrucciones
}
</script>
</head>
<body>
<button onclick="mi_funcion()">Click me</button>
</body>
</html>
<html>
<head>
<title>Invocar función desde JavaScript</title>
<script type="text/javascript">
function mi_funcion(){
       //instrucciones
}
</script>
</head>
<body>
       <button onclick="mi_funcion()">Click me</button>
</body>
</html>
```

Lógicamente las funciones también pueden llamar a otras funciones.

3.3 Consideraciones sobre funciones.

- Atención a la instrucción **return**

```
function sum(num1, num2) {
    return num1 + num2;
    alert("Hello world");    //nunca se ejecuta
}
```

- Los **argumentos** son un array

```
function howManyArgs() { // Se pueden omitir argumentos en la definición y
utilizarlos igualmente
    alert(arguments.length);  // prueba con for (i  hasta arguments.length y
visualizando arguments[i]
}

howManyArgs("string", 45);    //2
howManyArgs();                //0
howManyArgs(12);              //1
```

- Los parámetros se pueden pasar a las funciones de dos formas : por **valor** o por **referencia**. Un

parámetro que se pasa por **valor** a una función significa que se hace una copia del mismo y se envía a la función, con lo que si ésta modifica el valor, está modificando la copia y el original queda inalterado. Cuando se pasa un parámetro por **referencia**, lo que se envía a la función es la dirección de memoria donde se encuentra esa variable, con lo que si se modifica en la función la variable, ésta queda modificada en el original.

En **Javascript**, los argumentos se pasan por valor, no por referencia, con lo cual no es posible modificar el argumento dentro de la función. Otra cosa es cuando se pasan objetos, que se pasa la referencia y se podría modificar :

```
function setName(obj) { // obj se pasa por referencia
      obj.name = "Nicholas";
}
var person = new Object();
setName(person); // al ser un objeto se pasa por referencia.
alert(person.name); //"Nicholas"
```

- El array se considera un objeto, con lo que se pasa como referencia a una función.

- Si una función está definida **dos** veces, vale la última de ellas.

```
function addSomeNumber(num){
    return num + 100;
}
function addSomeNumber(num) {
    return num + 200;
}
var result = addSomeNumber(100);    //300
```

- El operador **typeof** devuelve los siguientes resultados :

```
var miFuncion = new Function("5+2")
var forma="redonda"
var tamano=1
var hoy=new Date()

typeof miFuncion == 'function'
typeof forma == 'string'
typeof tamano == 'number'
typeof hoy == 'object'
typeof noExiste == 'undefined'
```

- Una función puede ser **anónima**, es decir, no tener un nombre. Por ejemplo :

```
var cuadrado = function(number) {return number * number};
```

```
var x = cuadrado(4) //x  contendrá 16
```

Se puede proporcionar un nombre con el objetivo de llamarse así misma :

var factorial = function fac(n) {return n<2 ? 1 : n*fac(n-1)};
print(factorial(3));

- La mejor "situación" para definir un conjunto de funciones es en el <head> ya que así nos aseguraremos de que al utilizar la función en el <body> ésta ya se encuentre definida.
Vale también definirla antes o después de su utilización si se encuentra dentro del mismo bloque <script>.
Si la función está definida en un bloque <script> y la utilizamos antes, en otro bloque <script> obtendremos un error.

Mas información ==> http://www.desarrolloweb.com/articulos/584.php

- Sobre el siguiente ejemplo comentaremos **ámbitos** de las variables.

```
	<script type="text/javascript">
	function add(num1, num2) {
		sum = num1 + num2;  // Definida como local pero es global por no
poner "var"
		return sum;
	}
	var result = add(10, 20); //30
	document.write("<br> variable sum = "+sum); //30  ==> Definida como local
pero es global por no poner "var"

	var color = "blue"; // Dentro de getColor reconocen la variable
	function getColor(){
		document.write("<br> Dentro de la funcion 1 : " + color);
		color = "red";  // Se está refiriendo a la variable externa color,
con lo que modifica su valor
		document.write("<br> Dentro de la funcion 2 : " + color);
		return color;
	}
	document.write("<br> getColor() = "+getColor()); //"red"
	document.write("<br>Color = " + color);
	document.write("<br>---------------------------<br>");
	var tama = "grande";
	function getTama(){
		var tama = "peque";  //Crea una variable local. Existe solo dentro
de la función
		document.write("<br> Dentro de la funcion : " + tama);
		return tama;
```

```
      }
      document.write("<br> getTama() = "+getTama()); //"peque"
      document.write("<br>Tama = " + tama); // es el tama "global" porque la
función crea otra variable distinta
</script>
```

- Sobre el siguiente ejemplo vemos otra forma de dar valores por defecto a parámetros :

```
function multiply(a, b = 1) {
 return a * b;
}

console.log(multiply(5, 2)); ==> Salida = 10


console.log(multiply(5));   ==> Salida = 5
```

4. Arrays

Supongamos que tenemos que guardar en memoria los nombres de 30 productos. Podemos hacer :

var producto1 = “pan”;
var producto2 = “agua”;
var producto3 = “cereales”;
…..
luego, a la hora de mostrarlos :

document.write(producto1);
document.write(producto2);
…..
Un array es un conjunto ordenado de valores relacionados. Cada uno de estos valores se denomina elemento y cada elemento tiene un índice que indica su posición numérica en el array. Además de los valores y los índices de estos valores, un array debe tener un nombre.

De esta forma, en el ejemplo de los 30 productos, el array podría denominarse lista_productos :

lista_productos	
Índice	Contenido
0	pan
1	agua
2	cereales
….	….
30	mantequilla

4.1 Declaración de Arrays

Declaramos un array sin indicar su tamaño inicial ni su contenido :

```
var nombre_del_array = new Array();
```

Declaramos un array indicando su tamaño inicial :

```
var nombre_array = new Array(10);// Creamos array con 10 elementos. Del 0 al 9
```

Declaramos un array indicando su tamaño inicial y su contenido :

```
var lista_productos = new Array('Pan', 'Agua', 'Lentejas');
```

4.2 Inicializacion de Arrays

Si no se hizo al declarar el array se puede hacer de la siguiente forma :

nombre_del_array[i] = “valor”;

Donde i es el índice del array que deseamos inicializar. Ejemplo :

lista_productos[0] = “pan”;
lista_productos[1] = “agua”;
……

4.3 Arrays y bucles

La ventaja principal del array es poder utilizarlo mediante bucles. Veamos un ejemplo de inicialización de un array mediante un bucle y su posterior listado en pantalla.

```
var productos = new Array ();
for (var i=0; i<10;i++){
       productos[i] = "Codigo_producto_" + i;
}
var productos = new Array ();
for (var i=0; i<10;i++){
productos[i] = "Codigo_producto_" + i;
}
// Ahora listamos
var i;
for (i=0; i<10; i++) {
document.write (productos[i] + "<br>");
}

var i;
for (i=0; i<10; i++) {
       document.write (productos[i] + "<br>");
}
```

4.4 Propiedades de los Arrays

En Javascript un array es un objeto como los que hemos estudiado hasta ahora. Como objeto tiene sus propiedades y métodos. Las propiedades son :

length y **prototype**

- La propiedad **length** nos da el tamaño del array, siendo muy útil esta propiedad para recorrer un array.

```
var i;
for (i=0; i< productos.length ; i++) {
      document.write (productos[i] + "<br>");
}
```

Esta propiedad **no** es solo de lectura, pudiendo eliminar o añadir elementos modificando esta propiedad. Ejemplos :

```
var colors = ["red", "blue", "green"];     //Crea un array con 3 cadenas
colors.length = 2;
alert(colors[2]);          //undefined

var colors = ["red", "blue", "green"];     //Crea un array con 3 cadenas
colors[colors.length] = "black";           //añade un color (posicion 3)
colors[colors.length] = "brown";           //añade otro color (posicion 4)

var colors = ["red", "blue", "green"];     //Crea un array con 3 cadenas
colors[99] = "black";                      //añade un color (posicion 99)
alert(colors.length);                      //100
```

- **prototype** es una propiedad mas compleja y sirve para agregar nuevas propiedades y métodos al objeto Array. Veamos un ejemplo :

```
<html>
<head>
</head>
<body>
      cuerpo
      <script type="text/javascript">
            // Array y prototype
            var a = new Array("uno","dos","tres");
            Array.prototype.dia = "lunes";  // genera una propiedad para todos
los arrays. Ver depurador.

            var b = new Array(6, 4, 5);
            b.dia = "martes";  // crea una propiedad dentro del array b. Ver
depurador

            alert( " b.dia = " +  b.dia ) ;  // martes
            alert( " a.dia = " +  a.dia ) ;   // lunes

            // Añadimos el método Mayus al objeto Array
            // Para pasar el contenido de todo el array a mayúsculas
            Array.prototype.Mayus = function() {
```

```
                for (var i = 0; i < this.length; i++) {
                        this[i] = this[i].toUpperCase();
                }
        }
// Llamamos al nuevo método
a.Mayus();
// Listamos el contenido a ver como queda
for (var i = 0; i < a.length; i++) {
        document.write("a["+i+"] = " + a[i] + "<br>");
}
// Creamos otro método que sustituye un valor inicial por otro final
Array.prototype.Susti = function(vi, vf) {
        for ( var i = 0; i< this.length; i++) {
                if ( this[i] == vi ) {
                        this[i] = vf ;
                }
        }
}

var tabla = new Array("vino","cerveza","licor");
document.write("El elemento 0 de tabla es : "+ tabla[0] + "<br>" );

tabla.Susti( "vino", "agua");
document.write("Ahora el elemento 0 es : "+ tabla[0] );

    </script>
</body>
</html>
```

4.5 Métodos de los arrays

Método	Descripcion	Sintaxis
concat()	Une dos o mas arrays.	var tablatotal = tabla1.concat(tabla2);
indexOf()	Busca un elemento en el array y devuelve su posición. Busca desde el principio y a partir de la primera posición si no se especifica nada.	var a = frutas.indexOf("Kiwi"); // -1 si no lo encuentra

join()	Junta los elementos de un array en una cadena con un separador – opcional.	var fruta = ["Kiwi", "Limon", "Otra"];
		var energy = fruits.join();
		// Kiwi, Limon, Otra

lastIndexOf()	Busca un elemento en un array y devuelve su posición. Busca desde el final.	Similar a indexOf()
pop()	Borra el último elemento del array y devuelve su contenido.	var a = frutas.pop(); // “Otra” y el array queda con 2 elementos.
push()	Añade nuevos elementos al array y devuelve su nueva longitud.	a = frutas.push(“Mas”);
reverse()	Invierte el orden de los elementos en el array.	frutas.reverse();
shift()	Elimina el primer elemento del array y devuelve el elemento.	a = frutas.shift()

slice()	Selecciona parte de un array y devuelve su contenido.	var frutas = ["Banana", "Orange", "Lemon", "Apple", "Mango"];
		var citrus = fruits.slice(1, 3);
		// citrus = “Orange”,”Lemon”
		Desde 1 (“Orange”) hasta 3 sin incluirlo (“Lemon”)

sort()	Ordena los elementos del array	Mas información a continuación
splice()	Añade/elimina elementos de un array	Mas información : http://www.w3schools.com/jsref/jsref_splice.asp
toString()	Convierte un array en string y devuelve el resultado (entre comas)	frutas.toString(); // “Banana, Orange,Lemon, Apple, Mango”

unshift()	Añade elementos al inicio del array y devuelve la nueva longitud	var frutas = ["Banana", "Orange", "Apple"];
		frutas.unshift("Lemon","Pineapple");
		//Lemon,Pineapple,Banana,Orange,Apple

split()	Convierte un string en un array	var str =”lunes,martes,miercoles”; var dias = str.split(“,”) //dias[0]=”lunes”, dias[1]=”martes”, etc

Ampliamos el método sort() :

array.sort(funcionComparacion) ;
funcionComparacion ==> Opcional. Una función que define un ordenado alternativo. La función debe de retornar un valor negativo, cero o positivo dependiendo de los argumentos. Algo así como :

function(a, b) { return a-b }

Cuando el método sort() compara dos valores, envía los valores a comparar a la función comparación y ordena los valores dependiendo del valor retornado. Ejemplo :

1) Cuando se comparan los valores 30 y 100, el método sort() llama a la función comparación definida.
2) La función calcula 30 – 100 (se supone que la función indica return a-b) y devuelve -70 (valor negativo).
3) La función sort() tomará 30 como valor inferior a 100.

Ejemplo :

```
<script type="text/javascript">
var points = [40, 100, 1, 5, 25, 10];
var i ;
document.write("<br> Antes <br>");
for( i =0  ; i< points.length; i++) {
      document.write("puntos = "+ points[i] );
}
document.write("<br>Ordenando b-a <br>");
points.sort(function(a, b){ document.write("a - b "+a+"-"+b+" ==> "+ (Number(b)
- Number(a)) + "<br>"); return b-a});
document.write("<br> Resultado <br>");
for( i =0  ; i< points.length; i++) {
      document.write("puntos = "+ points[i] );
}
document.write("<br> Ordenando a-b <br>");
points.sort(function(a, b){ document.write("a - b "+a+"-"+b+" ==> "+ (Number(a)
- Number(b)) +"<br>"); return a-b});
document.write("<br> Resultado <br>");
for( i =0  ; i< points.length; i++) {
      document.write("puntos = "+ points[i] );
}
</script>
```

La salida del código anterior sería :

```
Antes
puntos = 40puntos = 100puntos = 1puntos = 5puntos = 25puntos = 10
```

```
Ordenando b-a
a - b 40-100 ==> 60
a - b 40-1 ==> -39
a - b 5-25 ==> 20
a - b 5-10 ==> 5
a - b 25-10 ==> -15
a - b 1-25 ==> 24
a - b 100-25 ==> -75
a - b 40-25 ==> -15
a - b 1-25 ==> 24
a - b 1-10 ==> 9
a - b 1-5 ==> 4

Resultado
puntos = 100puntos = 40puntos = 25puntos = 10puntos = 5puntos = 1
Ordenando a-b
a - b 100-40 ==> 60
a - b 100-25 ==> 75
a - b 40-25 ==> 15
a - b 10-5 ==> 5
a - b 10-1 ==> 9
a - b 5-1 ==> 4
a - b 100-1 ==> 99
a - b 25-1 ==> 24
a - b 25-5 ==> 20
a - b 25-10 ==> 15

Resultado
puntos = 1puntos = 5puntos = 10puntos = 25puntos = 40puntos = 100
```

4.6 Arrays multidimensionales.

Hasta ahora hemos visto arrays de una dimensión, es decir, solo hace falta un índice para acceder a un valor del array. Es posible crear arrays de mas dimensiones para ello podemos definir un array que contiene a su vez otros arrays en cada una de sus posiciones.
El array bidimensional es el array multidimensional que mas se utiliza en programación. Podemos pensar en ellos como una tabla de filas y columnas, con lo cual necesitaremos conocer dos índices para acceder a cada elemento. Por ejemplo :

[filas, columnas]	0	1	2
0	Elemento[0,0]	Elemento[0,1]	Elemento[0,2]
1	Elemento[1,0]	Elemento[1,1]	Elemento[1,2]
2	Elemento[2,0]	Elemento[2,1]	Elemento[2,2]

El siguiente ejemplo crea y lista un array bidimensional :

```
	<script type="text/javascript">
	// Ejemplo array bidimensional
	// Necesitamos almacenar por cada día de la semana una temperatura que se
produce cada 6 horas. Es decir, 4 valores por día.
	// Una solución es con un array bidimensional. Donde las filas son los
días de la semana y las columnas las temperaturas.
	// En el elemplo cubriremos los datos con valores aleatorios.
	var dias = new Array(7); // Supongo 0 Domingo a 6 Lunes
	// Cubrimos el array
	var i,j;
	for (i = 0 ; i< dias.length ; i++) {
		dias[i] = new Array(4); // Cada día contiene un array de 4 elementos
	}
	// Cubrimos los datos de forma aleatoria
	for(i = 0; i< dias.length; i++) {
		for(j = 0; j < dias[i].length ; j++) {
			dias[i][j] = Math.floor( Math.random() * 40);
		}
	}

	document.write("<br> Vemos el resultado  <br>");
	for(i = 0; i< dias.length; i++) {
		for(j = 0; j < dias[i].length ; j++) {
			document.write( "Temperatura dia "+i + " hora "+j + " =
"+dias[i][j] + "<br>");
		}
		document.write("<br>-------------------------------<br>");
	}
	</script>
```

5. Objetos definidos por el usuario

Hasta ahora hemos visto objetos predefinidos del lenguaje Javascript como el objeto window (objeto mas alto en la jerarquía), document o los Arrays, que también son objetos. Es posible crear nuestros propios objetos con sus métodos y propiedades.

En Javascript hay diversas formas de crear objetos, todas son válidas y sirven para cumplir diversos propósitos. Javascript está basado en prototipos y no en clases (como Java o C#). Veremos:

- Objetos simples.
- Objetos literales.
- Funciones como constructores.
- Herencia.

5.1 Objetos Simples

La forma más simple de crear un objeto es instanciar un objeto de tipo **`Object`** y agregarle las propiedades y métodos dinámicamente:

```
var persona = new Object();
persona.nombre = "Isaac";
persona.edad = 28;
persona.imprimir = function() {
  alert(this.nombre + ' ' + this.edad);
};

persona.imprimir(); // Mostrará un alert "Isaac 28"
```

Cuando agregamos propiedades de forma dinámica, estas solo se agregan al objeto en cuestión y nuevos objetos instanciados no tendrán estas propiedades:

```
var persona = new Object();
persona.nombre = "Isaac";

var persona2 = new Object();

alert(persona.nombre); // Muestra "Isaac"
alert(persona2.nombre); // Muestra "undefined"
```

5.2 Objetos Literales

En un objeto literal las propiedades son creadas como lista de pares key/value.

```
var persona = {
  nombre: "Isaac",
  edad: 28,
  imprimir: function() {
      alert(this.nombre + " " + this.edad);
  }
};

persona.nombre = "Isaac Zepeda"; // Puedo modificar sus propiedades
persona.imprimir(); // E invocar sus métodos
```

5.3 Funciones como constructores

Cuando construimos objetos simples o literales no hay manera de crear nuevas o varias instancias de estos objetos, digamos que necesito un objeto **`Persona`** y crear varias instancias de él, aquí es donde entran las funciones como constructores:

```
1  function Persona() {
2    // Para definir variables de instancia usamos la palabra reservada 'this'
3    this.nombre = "Isaac";
4    this.edad = 28;
5
6    // Esta es una variable local a la función y no puede ser accedida desde fuera.
7    var count;
8  }
9
10 // Para instanciar un nuevo objeto usamos la palabra reservada new
11 var persona1 = new Persona();
12 var persona2 = new Persona();
13 alert(persona1.nombre); // Isaac
14
15 persona2.nombre = "Pancho";
16 alert(persona2.nombre); // Pancho
17
18 alert(persona1.count); // undefined, no se puede acceder ya que es una variable
   local no de instancia
```

Ahora podemos pasarle valores al constructor para que establezca **`nombre`** y **`edad`** con el valor que nos plazca, y si no están definidos que tome un valor por default usando el operador **`||`**.

```
1 function Persona(nombre, edad) {
2   // si nombre es indefinido o nulo tomará el valor "Isaac"
3   this.nombre = nombre || "Isaac";
4   this.edad = edad || 28;
5 }
6
7 var p1 = new Persona("Alberto", 17);
8 var p2 = new Persona();
```

5.4 Agregando Métodos

Hay dos maneras de agregar métodos, la primera es agregarlos como cualquier otra propiedad dentro del constructor y la otra definiéndolo en su **`prototype`**.

Defiendo el método dentro del constructor

```
1 function Persona() {
2   this.nombre = "Isaac";
3
4   this.caminar = function() {
5       alert(this.nombre + " está caminando");
6   }
7 }
```

Definiendo el método es su prototype

```
1 function Persona() {
2   this.nombre = "Isaac";
3 }
4
5 Persona.prototype.caminar = function() {
6   alert(this.nombre + " está caminando");
7 }
```

Ambos funcionan correctamente, pero hay pequeñas diferencias.

Si se define el método dentro del constructor cada vez que se instancie el objeto creará una propiedad con un objeto de tipo función, así si el objeto **`Persona`** se instancia 3 veces se crearán 3 funciones en memoria que hacen lo mismo. En cambio si lo creamos en el **`prototype`** la función se creará solo una vez, y todas las instancias de **`Persona`** apuntarán a la misma función, usando menos memoria.

Sin embargo, cuando creamos un método en el **`prototype`**, este no puede acceder a las variables locales del constructor:

Accediendo a variables locales

```
1  function Persona() {
2    var count = 1;
3
4    this.caminar = function() {
5        alert(count); // Muestra 1;
6    }
7  }
8
9  Persona.prototype.correr = function() {
10   alert(count); // Dará error ya que count no esta definida
11 }
```

El **`prototype`** es un objeto que tienen todos los objetos en Javascript, cuando queremos acceder a una propiedad **`Persona.nombre`** Javascript busca la propiedad en el objeto mismo, si no lo encuentra lo busca en el **`prototype`**, si no lo encuentra busca dentro del **`prototype`** del objeto **`prototype`**, esto se conoce como *prototype-chain* y nos sirve para la herencia.

5.5 Herencia

A través del *prototype-chain* podemos tener herencia en Javascript.

Supongamos que tenemos un tipo padre **Mamifero** que tiene el método **caminar** y **ver**. Luego tenemos un hijo **Humano** que sobreescribe al método **caminar** y agrega un nuevo método **correr**.

Para heredar **Humano** de **Mamifero**, el objeto **prototype** de **Humano** debe contener una instancia de su padre, en este caso **Mamifero**.

Herencia

```
1  // Definimos al padre
2  function Mamifero() {
3    this.extremidades = 4;
4  }
5
6  Mamifero.prototype.ver = function() {
7    alert('El mamífero ve.');
8  }
9
10 Mamifero.prototype.caminar = function() {
11   alert('El mamífero camina con ' + this.extremidades + ' extremidades.');
12 }
13
14 // Definimos al hijo
15 function Humano() {
16   this.extremidades = 2;
17   this.nombre = "Isaac";
18 }
19
20 // Le decimos quien es su padre
21 Humano.prototype = new Mamifero();
22
23 // Reescribimos y agregamos los métodos de Humano
24 Humano.prototype.caminar = function() {
25   alert('El humano camina con ' + this.extremidades + ' extremidades.');
26 }
27
28 Humano.prototype.correr = function() {
29   alert('El humano corre');
30 }
31
32 var perro = new Mamifero();
33 var persona = new Humano();
34
35 perro.ver(); // Método definidio en Mamifero
36 perro.caminar(); // Llama al método en Mamifero
37
38 persona.ver(); // Llama al método definido en Mamifero
39 persona.caminar(); // Llama al método sobreescrito en Humano
```

```
40 persona.correr(); // Llama al método definido en Humano que no existe en Mamifero
```

El objeto **`Humano.prototype`** es un objeto de tipo **`Mamifero`** (línea 21), luego agregamos dinámicamente métodos a ese prototipo (líneas 24, 25). Como **`Mamifero`** es un objeto el también tiene un propiedad **`prototype`** al cual se le defineron 2 métodos (líneas 6, 10).

Entonces, cuando invocamos el método **`persona.caminar()`** Javascript busca en el objeto mismo, no lo encuentra y busca en su prototipo (**`Humano.prototype.caminar`**), lo encuentra y lo invoca.

Cuando invocamos el método **`persona.ver()`** Javascript busca en el objeto mismo, no lo encuentra, luego busca en su prototipo (**`Humano.prototype.ver`**), no lo encuentra, y se va en la cadena de prototipos hasta que lo encuentre en este caso **`Humano.prototype.prototype.ver`**, ya que **`Humano.prototype.prototype`** apunta a **`Mamifero.prototype`**.

5.6. Objetos json y arrays.

Veremos unos ejemplos de manejo de datos con objetos json y arrays. Una forma de obtener datos de un servidor al que hacemos una petición es obtener los datos en forma de objeto json. Veremos como obtener datos de un servidor con Ajax, por ahora vamos a **simular** que tenemos los datos en el script escribiéndolos en el propio script. Veamos unos ejemplos :

```
var O = {
        "nombre":"cámara fotos",
        "seccion": "fotografía",
        "precio":450
    }

console.log( O[“nombre”] ) ; // mostrará “cámara fotos”
console.log( O.nombre ) ; // mostrará “cámara fotos”
var a = “seccion”;
console.log( O[ a ] ) ; // mostrará “fotografía”
console.log( O.a ) ; // mostrará undefined (ya que no existe la propiedad “a”)
O.cantidad = 5; // añade una propiedad nueva. El objeto pasa a ser el siguiente :

{
        "nombre":"cámara fotos",
        "seccion": "fotografía",
        "precio":450,
        “cantidad” : 5
    }


for( var e in o ) {  // in hace que se recorran las propiedades de un objeto.
      console.log( e ); // e contendrá "nombre" "seccion"  "precio" y “cantidad”
      console.log(o[e]); // Mostrará : "cámara fotos " “fotografía” 450 y 5
}
```

Veamos otro ejemplo. Supongamos que tenemos el siguiente dato :

```
{     “nombre”:”José Luis”,
      “edad”:21,
      “curso”: “CS Informatica”,
      “Asignatura”: “Redes”,
      “Notas”: [4, 6, 7, 8];
}
```

Significado : Nombre del alumno, edad, curso al que pertenece, asignatura de la que está matriculado (para simplificar suponemos de 1 sola siempre) y notas de esa asignatura (tantas como

exámenes vaya haciendo, no tiene porqué coincidir con la evaluación).
Crearemos un array de alumnos de la siguiente forma :

```
var alumnos = [ {  “nombre”:”José Luis”,   // Observa la apertura y cierre con []
      “edad”:21,
      “curso”: “CS Informatica”,             // cada objeto entre llaves {}
      “Asignatura”: “Redes”,
      “notas”: [4, 6, 7, 8];                 // esta propiedad contiene un array
},                                           // El array tiene varios tamaños.
{     “nombre”:”Marta”,
      “edad”:24,
      “curso”: “CS Informatica”,
      “Asignatura”: “AO”,
      “notas”: [4, 7, 8];
},
{     “nombre”:”José Luis”,
      “edad”:31,
      “curso”: “CS DAW”,
      “Asignatura”: “Redes”,
      “notas”: [4, 5, 6, 7, 8];
},
{     “nombre”:”Ana”,
      “edad”:19,
      “curso”: “CS DAW”,
      “Asignatura”: “DWCC”,
      “notas”: [4, 6, 2, 1 ,3, 7, 8];
} ];
```

Vamos a ordenar el array por el campo curso.

```
alumnos.sort( ordenar( a, b ) ) ;

function ordenar( a, b ) {
      if( a.curso > b.curso) {
            return 1;
      }
      if( a.curso < b.curso) {
            return -1;
      }
      return 0;
}
```

Vamos a calcular la nota media de cada alumno, para ello vamos a recorrer el array de alumnos y, por cada alumno, el array de notas.

```
for(var e of alumnos) {  // for(var i = 0; i<alumnos.length ; i++) ==> también vale)
      // e = primer objeto
      // alumnos[i] = primer objeto ( si se hace el otro for)
      console.log( “alumno  : “ + e.nombre ) ; // alumnos[i].nombre ==> otra opción
      var suma = 0;
      for( var nota of e.notas ) { // for( var j = 0; j<alumnos[i].notas.length; j++)
            suma = suma + nota;
            // suma = suma + alumnos[i].notas[j];
      }
      var media = suma / e.notas.length //  alumnos[i].notas.length
      console.log( “nota media “ + media);
      // Si quisieramos añadir una nueva propiedad con su nota media :
      e.notamedia = media ; // añadimos la propiedad “notamedia” a cada objeto.
}
```

IMPORTANTE : Cuando a una función se le pasa como parámetro un array o un objeto, le estamos pasando el propio array u objeto, no una copia, es decir, por referencia. Cualquier cambio que se realize en el array u objeto modifica el objeto original.

```
Ejemplo :

var alumno = { “nombre”:”José Luis”,
      “edad”:21,
      “curso”: “CS Informatica”,
      “Asignatura”: “Redes”,
      “Notas”: [4, 6, 7, 8];
};

var rdo = consulta( alumno ) ;  // alumno.edad = 23 al ejecutar la función.

function consulta( a ) {
      var resul = true;
      a.edad = 23;  // Modifica el objeto
      return resul;
}
```

5.7 Ejemplos

1) Crearemos un objeto que únicamente tenga datos. Pares key/value, clave / valor. **DatosTipoJson.html** (Este ejemplo es el mismo visto anteriomente)

```
var o1 = {
      "nombre" : "Jose Luis",
      "edad" : 31 ,
      "notas" : [4,6,7 ]
}
```

```
let o2 = {
      "nombre" : "Antonio",
      "edad" : 21 ,
      "notas" : [6,8,7, 4 ]
}
let o3 = {
      "nombre" : "Ana",
      "edad" : 42 ,
      "notas" : [5,7,9,6,4 ]
}
alert( o1.nombre ) ; //      ==> Muestra "Jose Luis"
alert( o1["nombre"] ) ;
// ==> Muestra "Jose Luis". Es interesante esta última opción, ya que
podemos sustituir lo que va entre corchetes por una variable. Ejemplo :

var dato = "nombre";
alert( o1[dato] ) ; // Muestra "Jose Luis"
// Sin embargo no funcionaría :
// alert( o1.dato ) ; ==> busca una propiedad llamada "dato" y no la
encontraría.

// Recorremos un objeto por sus propiedades, es decir, obtenemos el valor
de cada propiedad

for( var e in o1 ) {
      console.log( e ); // e contendrá "nombre" "edad" y "notas"
      console.log(o1[e]); // Mostrará : "Jose Luis" 31 y [4,6,7]
}
console.log(o1["notas"][1]);   // Mostrará 6. Ya que o["notas"] es un
array (ver punto anterior), muestra el elemento con índice 1, es decir,
el segundo.

var lista = [ o1, o2, o3];

// Como podemos ordenar el array lista ?
// Lo listamos antes de ordenar
for(var elemento  of lista) {
// elemento = cada elemento de lista, o sea cada objeto.
// No confundir con : for ( var a in objeto)
       console.log( elemento ); // Mostramos todo el objeto.
```

```
      console.log( elemento.nombre ) ; // Mostramos únicamente el nombre
de cada elemento :
}
function ordenar(o1, o2) {
    return o1.edad - o2.edad;
}
lista.sort(ordenar);   // ordenamos por edad. Como sería ordenar por
nombre ?
// Lo listamos despues de ordenar
for(var elemento  of lista) {
      console.log( elemento.nombre ) ;
}

// Como calculamos la media de edad de cada alumno ?

for(var elemento  of lista) {  // Por cada elemento de la lista
     var suma = 0;
     for(var n of elemento.notas) {   // n es cada una de las notas
          suma = suma + n;
     }
     var media = suma / elemento.notas.length;
     console.log("El  alumno  :  "+elemento.nombre+"  tiene  media  de  :
"+media);
}
```

2) Los datos los clasificamos (modificamos). **Clasificar.html**

```
var lista= [
    { "nombre": "Alex", "edad": 20, "curso": "A" },
    { "nombre": "Andrea", "edad": 18, "curso": "A" },
    { "nombre": "Luisa", "edad": 22, "curso": "B" },
    { "nombre": "Juan", "edad": 16, "curso": "C" },
    { "nombre": "Jose", "edad": 21, "curso": "D" },
    { "nombre": "Peter", "edad": 15, "curso": "A" },
    { "nombre": "Blanca", "edad": 22, "curso": "B" },
    { "nombre": "Lucia", "edad": 22, "curso": "D" },
]
// Como podemos obtener la media de edad de cada curso ?
// No sabemos cuantos cursos distintos existen.
// Una opción es reordenar la presentación de los datos, clasificarlos.
```

```
// Todos los del curso A en un array, luego los del curso B, etc
// Podemos realizar lo siguiente :

function clasificar( c ){
    var lista = {};
    for(var elemento of c){
        if(lista[elemento.curso]==undefined){
            lista[elemento.curso]=[];
        }
        lista[elemento.curso].push(elemento);
    }
    return lista;
}

var lista2 = clasificar( lista );
console.log( lista2); //IMPORTANTE entender los datos que generamos.
// Ahora calcular la media es trivial.
for( var elemento in lista2) { // Recorro cada una de las propiedades
      var suma=0;
      for(var ele of lista2[elemento] ) { //ele es un array !!
            console.log("Edad "+ele.edad );
            suma = suma +ele.edad;
      }
      console.log("La media del curso "+elemento+" es : " +
suma/lista2[elemento].length);
}
// La función clasificar crea un nuevo objeto donde los datos se
clasifican por curso. Modifica la función de forma que le indiquemos, por
medio de un parámetro, la propiedad por la que que deseamos clasificar.
```

3) Crearemos un objeto simple que maneje una máquina tragaperras. **EjemploFuncionTragaperras.html**

```
function tragaperras() {
            this.saldo = 10;  // Saldo del jugador. 10 por defecto
            this.puntos = 0;   // Lo que va ganando. 0 por defecto.
            this.imprimir =  function() {
                  document.write("<br> Tengo de saldo : "+this.saldo+" y
"+this.puntos+" puntos <br>");
            }
```

```
            // Método jugar
            this.jugar = function() {
                  document.write("Jugamos    teniendo    el    saldo    :
"+this.saldo);
                  this.saldo--;
            }
            // Método calcularPuntos
            this.calcularPuntos = function() {
                  this.puntos = this.puntos + calculo();// ganamos siempre
            }
            function calculo(){
                  return 2;
            }
      }
      // Hacemos 10 tiradas y vemos los resultados
      var traga = new tragaperras();
      do  {
            traga.jugar();
            traga.imprimir();
            traga.calcularPuntos();
      } while ( traga.saldo > 0 );  // No es bucle infinito puesto que a
saldo no se le suman los puntos
console.log( "a ver que pasa : "+traga.calculo()) ; // No la reconoce
```

6. Clases en javascript

Veamos un ejemplo práctico de una clase en javascript.

```
class Rectangulo {
        constructor(alto, ancho){
          this.alto = alto;
          this.ancho = ancho;
        };
        area(){  // Un método que devuelve algo
              var x =this.alto * this.ancho ;
          return x;
        };
        get damealto() { // gracias al get se utiliza como si fuese una propiedad
              return this.alto;
        }
         damealto2() {
              return this.alto;
         }
        set ponalto(x) { // gracias al get se utiliza como si fuese una propiedad
              return this.alto = x;
        }
         ponalto2(x) {
              return this.alto = x;
         }
       static instrucciones(){ // método estático no hace falta instanciarlo
            document.write("Texto diverso para probar el método ");
          }
};

let r = new Rectangulo(2,3);  // Creamos una instancia
document.write(r.area()); //6
document.write(" alto = "+r.alto); // Accedemos directamente a la variable. Se puede evitar con closures.
document.write(" dame alto = "+r.damealto  ); // llamo a la función como si fuese una propiedad

document.write(" dame alto2 = "+r.damealto2()  ); // No funciona sin los paréntesis
```

```
r.ponalto = 6;        // Modifica el alto con el set
document.write(" dame alto nuevo  = "+r.damealto  ); // llamo a la función como si fuese una propiedad
r.ponalto2(7);        // Modifica el alto sin el set
document.write(" dame alto nuevo 2  = "+r.damealto  ); // llamo a la función como si fuese una propiedad
Rectangulo.instrucciones(); // Llamamos al método sin instanciar ya que es estático
```